

Instruction Emulation and OS Supports of a Hybrid Binary Translator for x86 Instruction Set Architecture

I-Chun Liu, I-Wei Wu, Jean Jyh-Jiun Shann

Dept. of Computer Science
National Chiao Tung University
Hsinchu, Taiwan
e-mail: jjshann@cs.nctu.edu.tw

Abstract—Binary translation is one of the most important techniques of virtualization. The main purpose of a binary translator (BT) is to translate an executable from a source instruction set architecture (ISA) to a target ISA. Traditionally, there are two types of binary translators: static binary translator (SBT) and dynamic binary translator (DBT). In recent years, a new type of BT called hybrid binary translator (HBT) was proposed, which translates the source executable first at static time, and then, at run time, if the execution of the target executable emits an exception because of reaching statically untranslated code, it switches to the attached dynamic translator for translating these code. Therefore, an HBT may have the merits of both good performance of SBT and easy handling of code discovery and code location problems of DBT. Nowadays, massive application programs have been developed for x86 platforms, and thus, many binary translators have been proposed for x86 ISA. However, due to the characteristics of CISC architecture of x86, for example, variable-length instruction format, the BT designed for it previously usually apply dynamic translation strategy or make use of profiling data for resolving the code discovery and code location problems. In this paper, we present an HBT which supports x86 ISA and emulates the execution behavior of an x86 executable under Linux operation system. In our x86-32 to x86-64 translation experiments, the target executables translated by our HBT outperform that of QEMU on most programs of EEMBC benchmark suite.

Keywords- virtualization; static binary translation; dynamic binary translation; hybrid binary translation; compiler

I. INTRODUCTION

Binary translation is a popular technique in computer science that has been researched and developed in recent twenty years [1, 2]. A binary translator (BT) can be used for the migration of an executable from one ISA to another ISA, and then it can be applied in many research areas, such as same-ISA binary optimization, binary profiling, and process/system virtual machine [3-6].

Traditionally, BTs can be classified into two categories, static binary translator (SBT) and dynamic binary translator (DBT) [6, 7]. An SBT translates the source executable with aggressive optimizations at static time. After the target executable is generated, we can run it with high performance. However, for an SBT, it is hard to handle some problems related to runtime behaviors of the source

executable, for examples, code discovery problem, code location problem, and Self-modifying code.

A DBT translates the source executable and executes the translated code at runtime [6, 7]. Basically, there are a code cache and an address mapping table in a DBT for the concern of performance. The code cache is used to reserve the translated blocks, and the address mapping table is used to record the pairs of the source ISA instruction addresses and their corresponding target ISA instruction addresses. First, a DBT will check the address mapping table to see whether the next translation block is translated or not. If the next translation block has already been translated, the DBT will execute the translated block in the code cache directly; otherwise, it will translate the block before execute it. Once there is a new block being translated, the translated block in target ISA will be reserved in the code cache, and a pair of addresses including the source program counter and the corresponding target program counter will be inserted into the address mapping table. A DBT has the merits of handling the code discovery problem, code location problem, and self-modifying code efficiently. However, a DBT does not perform aggressive optimizations since the translation time is included in runtime. Furthermore, since a DBT requires a code cache and an address mapping table, it might cause huge memory usage.

Recently, a new BT called hybrid binary translator (HBT) has been proposed [8]. An HBT translates the source executable to the target executable and links the target executable with the source image and a dynamic binary translation library at static time. While the target executable is in execution, if a branch target address cannot be found in the address mapping table of the HBT constructed statically, the execution of the target executable will switch the control to the dynamic binary translation library. After that, the dynamic binary translator will translate the source instructions from the source image again at runtime. Moreover, the HBT may switch back to the statically translated executable if the address of the next translation block is found in the address mapping table. An HBT has both the merits of an SBT and a DBT since it has both good performance and the ability of handling the code discovery problem, code location problem, and self-modifying code.

Presently, the LLVM-based retargetable HBT proposed by Shen et al is the only example of HBT [8]. This HBT use LLVM Machine Code (MC) toolkit to disassemble the

source ARM executable to get the corresponding LLVM MC intermediate representation (IR) and then translates the MC IR into LLVM IR. Since this HBT is developed based on LLVM, it is retargetable. However, this HBT supports ARM, an RISC, source ISA only.

x86 is a popular CISC architecture and massive applications have been developed for x86 platforms. Therefore, binary translation of x86 source executable is in demand, and several binary translators which support x86 source ISA have been designed and implemented [9-12]. However, due to the code location and code discovery problems encountered for the binary translation of a CISC ISA, these binary translators usually apply dynamic translation strategy or make use of profiling data for translating x86 source executable.

In this paper, we extend the HBT proposed by Shen et al to support the binary translation of x86 source executable. Compared to ARM source ISA, there are several challenges of extending the HBT to support x86 source ISA. First, the instruction formats of x86 ISA are much more complicated, for examples, its instruction length is variable and it has sub-registers. Second, there are many more instructions in x86 ISA. Third, the program behavior of an x86 executable is more complicated since the operation system (Linux) has special optimizations on it, for example, fast system call.

The rest of this paper is organized as follows: Section II presents the related work and software development tools of our design. Section III describes the design and implementation issues of our work. In Section IV, we show the experiment results and analysis. Finally, we conclude our work in Section V.

II. RELATED WORK AND DEVELOPMENT TOOLS

In this section, we will introduce the related work of our design and describe the software development tools used in our work, including LLVM, LLVM JIT compiler, and LLVM MC Toolkits.

A. Related Work

QEMU is a retargetable and efficient dynamic binary translation system [11]. It supports many source ISAs, as well as target ISAs. In QEMU, the emulation of a source executable is divided into two phases. First, QEMU translates each basic block of the source instructions into a sequence of micro-operations by its tiny code generator (TCG). Here the micro-operations are called TCG intermediate representation (TCG-IR). Second, QEMU will convert the TCG-IR into the related native codes. QEMU maintains a table that maps each micro-operation to a sequence of native codes compiled at static time, so there is no compilation overhead but negligible overheads from converting TCG-IR into native codes. Once a translated basic block is generated, it can be run on target machine. In QEMU, there are several optimizations made for improving the emulation performance, such as condition codes optimizations, block chaining, and etc. However, since it is a dynamic binary translator, the translation time is included as runtime overheads.

The binary translator proposed by Shen et al is an LLVM based retargetable hybrid binary translator for ARM architecture [8]. It is the first hybrid binary translator which may perform aggressive optimizations and handle the problems related to indirect branch efficiently in the same time. For a source executable, it first performs the static binary translation which disassembles the source executable into LLVM MC IR, translates the LLVM MC IR into LLVM IR, and then, optimizes, assembles, and compiles the LLVM IR file into target object file. Next, the target object file is linked with the source image and the runtime system. The runtime system is composed of a dynamic translator and system call helper functions. When the target executable is executed, it first initializes the runtime system, and then executes from the static translated program entry point. If the execution encounters an indirect branch, it will check whether the branch target is in the address mapping table or not. If the branch target address is found in the address mapping table, it will jump to the target and execute the statically translated instructions. Otherwise, it will switch to the dynamic translator which may translate the instructions and executes them or just execute the dynamically translated instructions in its code cache according to whether the instructions have already been dynamically translated and saved in the code cache or not. This HBT successfully combines the merits of SBT and DBT.

B. Software Development Tools

1) LLVM

LLVM is a compiler framework designed by Vikram Adve and Chris Lattner to support transparent, life-long program analysis and transformation for arbitrary programs in 2000 [13]. Since LLVM is a collection of modular and reusable compiler and toolchain technologies, it can be reused for the different development requirements. Sub-projects developed under LLVM include Clang, LLDB, dragonegg, LLVM-MC toolkit, and so on [14-17]. In this work, we will use LLVM static compiler, LLVM-JIT and LLVM-MC toolkit to implement our HBT.

LLVM intermediate representation (LLVM IR) is an SSA-form machine independent intermediate representation. LLVM can convert the source code of supported languages such as C/C++ into LLVM IR, and then convert the LLVM IR into a target assembly file. While in LLVM IR stage, we may use optimizations provided in LLVM to obtain optimized LLVM IR.

2) LLVM-JIT

The LLVM-JIT compiler is contained in LLVM Execution Engine. It may be used to compile a sequence of LLVM in-memory IR. After the compilation, the LLVM-JIT compiler will return the starting address of the compiled target binary, and then, we may execute the target binary from the starting address. Since the LLVM-JIT compiler is processed at run-time, it may perform optimizations on the in-memory IR according to the run-time environment. However, it may not perform too aggressive optimizations for the consideration of performance.

3) LLVM-MC toolkit

LLVM-MC toolkit is a sub-project of LLVM and is being created to solve the problems under CPU instruction level, such as assembly, disassembly, object file format handling, and so on [17]. We may use LLVM with LLVM-MC toolkit to compile a program without using an external assembler which might have a lot of bugs and lack of modern features. Furthermore, it saves the compilation time since an external assembler needs to lexical analyze, parse, and validate the assembly file. In the LLVM-MC definition, every instruction from different ISA is well defined, called as an LLVM MC instruction (MCInst) or LLVM MC IR, so that we can use the disassembled MCInst with detailed information.

III. DESIGN AND IMPLEMENTATION

The goal of our research is to design and implement an HBT which may support the x86 source ISA. In the following subsections, we will first describe the system overview of our HBT and then the main instruction translation issues and OS supports for x86 ISA.

A. Overview of Our HBT for x86 ISA

The static translation flow and dynamic execution flow of our HBT for x86 ISA are similar to that of the HBT proposed by Shen et al [8].

1) Static binary translation

The flow of static binary translation is shown in Figure 1 and the translation process is described as follows:

- Each instruction in the input source executable is disassembled into an LLVM MCInst by the LLVM MC disassembler, and then, each disassembled LLVM MCInst will be translated into LLVM IR which emulates the behaviour of the original instruction. After all instructions are disassembled and translated, the generated LLVM IR will be saved in a file. In the same time, a linker script and a source image are also generated. The linker script is used for specifying the memory layout of the target executable. The source image will be combined with

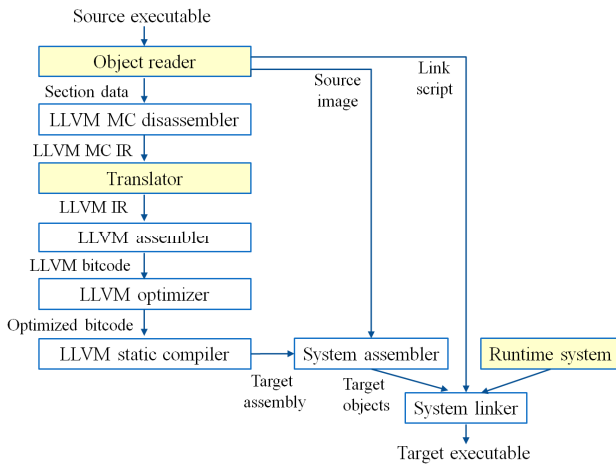


Figure 1. The translation flow of the static binary translation in our HBT.

the target executable by the system linker and is used for re-translation at dynamic time. While performing the static translation, an address mapping table is maintained for preserving the following information: the return address of a function call and the address of instruction that follows a return instruction.

- The generated LLVM IR is assembled to LLVM bitcode by the LLVM assembler.
- The LLVM bitcode is optimized by the LLVM optimizer.
- The optimized LLVM bitcode is compiled to target assembly by LLVM static compiler.
- The target assembly and the source image are assembled into target objects.
- In the final step, according to the linker script, the files including the source image, the translated and assembled target object, and the runtime system are linked into the target executable by the system linker.

2) Execution flow

After the target executable is produced, it will be executed on the target machine. The execution flow is shown in Figure 2 and the execution steps are described as follows:

- The target executable first initializes the dynamic translator, and then, set up the emulated source stack. After that, the target executable copies the auxiliary vector, the environment variables, argument vector, and argument count from the stack of target executable into the emulated source stack.
- After initialization, the target executable starts execution from its emulated program entry point.
- If the instruction in execution is an indirect branch, the target executable will check the statically generated address mapping table which maps the source instruction addresses and the target instruction addresses. If the target address of the indirect branch is found in the address mapping table, the target executable will jump to the corresponding target address and continue its execution.
- If the branch target is not found in the address mapping table, the target executable will switch to the dynamic binary translator. After that, the dynamic binary translator will check whether the branch target has been translated and save in the

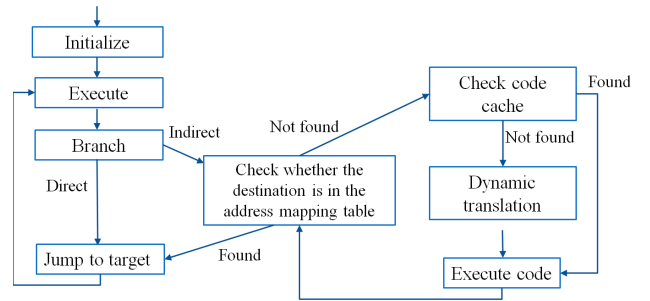


Figure 2. The Execution flow of a target executable.

code cache of the dynamic binary translator or not. If the branch target is found in the code cache, the translated code will be executed immediately. Otherwise, the dynamic binary translator will translate the instructions from the branch target in the source image, save the translated instructions in code cache, and finally execute the translated instructions. The dynamic binary translator will translate instructions from indirect branch to another indirect branch in order to enlarge the size of a translation block.

The details of the dynamic time translation in the HBT are described in the following:

- The dynamic binary translator of our HBT disassembles the source instruction into LLVM MCInst, and then translates the LLVM MCInst into LLVM IR. It will continuously disassembles and translates the instructions in the source image until an indirect branch is reached. After the translation, the translated LLVM IR instructions will be constructed as an LLVM function and the return value of the LLVM function is the destination address of the indirect branch.
- After the LLVM function is constructed, it applies the LLVM JIT to compile the function to a native function on target machine.

B. Translation of MC Instruction to LLVM IR

After we use LLVM MC disassembler to disassemble the x86 source executable, each x86 machine code is disassembled as an LLVM MCInst. For each original x86 instruction, it may be classified into multiple MCInsts according to the size and the types of its operands. The size of an operand could be 8, 16, and 32 bits, and the type of an operand could be a register number, a memory address or an immediate value. Furthermore, these operands could be a source operand or destination operand. Therefore, there are many combinations of LLVM MCInsts for an original x86 instruction. Each LLVM MCInst is represented by an opcode and its corresponding operands.

In our binary translator, we have implemented more than six hundreds of the translation functions for translating the LLVM MCInsts defined for the most commonly used x86 integer instructions in general executable to LLVM IR. By our implementation of these x86 integer instruction emulation, our hybrid binary translator may successfully translate most of the executables which is constructed by x86 integer instructions.

An example of translating LLVM MCInst *add32mr* to LLVM IR is shown in Figure 3. First, the x86 machine code is disassembled into an LLVM MCInst *add32mr* with an opcode and five operands. Then, the LLVM MCInst is translated into the corresponding LLVM IR by the translation function shown in Figure 4.

C. Sub-Register Emulation

In x86 ISA, there are four 32-bit general purpose registers called EAX, EBX, ECX, and EDX, and five 32-bit index and point registers called ESI, EDI, ESP, EBP, and

0x01 0x05 0x17 0x50 0x40 0x00
(a) x86 machine code of instruction “*add ds:0x405017,eax*”

```
-opcode : 76
-operand 0 : <MCOperand Reg:0>
-operand 1 : <MCOperand Imm:1>
-operand 2 : <MCOperand Reg:0>
-operand 3 : <MCOperand Imm:4214807>
-operand 4 : <MCOperand Reg:0>
-operand 5 : <MCOperand Reg:43>
```

(b) Disassembled LLVM MC IR “*add 32mr*”

```
L_00000330_1:                                ; preds = %L_00000330_
%2 = load i32* @inttoptr (i32 4214807 to i32*)
%3 = load i32* @EAX
%4 = call { i32, i1 } @llvm.uadd.with.overflow.i32(i32 %2, i32 %3)
%5 = call { i32, i1 } @llvm.sadd.with.overflow.i32(i32 %2, i32 %3)
%6 = extractvalue { i32, i1 } %4, 0
store i32 %6, i32* @inttoptr (i32 4214807 to i32*)
store i32 %6, i32* @Pf
%7 = lshr i32 %6, 31
store i32 %7, i32* @Sf
%8 = icmp eq i32 %6, 0
%9 = zext i1 %8 to i32
store i32 %9, i32* @Zf
%10 = extractvalue { i32, i1 } %4, 1
%11 = zext i1 %10 to i32
store i32 %11, i32* @Cf
%12 = lshr i32 %6, 4
%13 = and i32 %12, 1
store i32 %13, i32* @Af
%14 = extractvalue { i32, i1 } %5, 1
%15 = zext i1 %14 to i32
store i32 %15, i32* @Of
br label %L_00000330_2
```

(c) Translated LLVM IR

Figure 3. An example of translating LLVM MCInst to LLVM IR.

```
void X86Translator::transAdd32MR(const MachInst &inst)
Value *memVal, *op1;
Value *ures, *sres, *res;
Value *cout, *oflow, *neg, *zero, *Auxiliary;
loadmem(inst, 0, memVal, i32p);
op1 = load(reg(inst.getOperand(5)));
ures = call(uaddOverflow, memVal, op1, NULL);
sres = call(saddOverflow, memVal, op1, NULL);
res = extractvalue(ures, 0);
storemem(res, inst, 0, i32p);
store(res, reg(Pf));
neg = lshr(res, const31);
store(neg, reg(Sf));
zero = icmp_eq(res, const0);
zero = zext(zero, i32);
store(zero, reg(Zf));
cout = extractvalue(ures, 1);
cout = zext(cout, i32);
store(cout, reg(Cf));
Auxiliary = lshr(res, const4);
Auxiliary = zext(Auxiliary, i32);
Auxiliary = and2(Auxiliary, const1);
store(Auxiliary, reg(Af));
oflow = extractvalue(sres, 1);
oflow = zext(oflow, i32);
store(oflow, reg(Of));
```

Figure 4. The translation function of LLVM MCInst “*add 32mr*”.

EIP. All of these registers contain their own sub-registers. For example, the general purpose register EAX contains AX, AH, and AL sub-registers. AX is a 16-bit sub-register which contains the least significant 16 bits of EAX. AH and AL contain the most significant 8 bits and least significant 8 bits of AX, respectively.

For a binary translator, there are two possible methods to emulate these registers and sub-registers. The first method refers to declare all registers and sub-registers as independent variables. By this method, a binary translator needs to perform synchronization whenever a register or sub-register is modified. However, the synchronization mechanism may cause some unacceptable overheads. The second method refers to declare a 32-bit register as a variable and declare each of its sub-register as a pointer which points inside of the 32-bit register. By this method, after we set up the pointers of the sub-registers, whenever there is a modification of the register or any of the sub-register, the register and its sub-registers are always synchronized without extra mechanism. Therefore, we choose the second method as our solution for emulating sub-registers.

D. Status Flag Emulation

In x86, the 32-bit register called EFLAGS contains the flag information of the processor state. There are status flags, control flags, system flags and other reserved bits, and each flag is represented by one bit in the EFLAGS register. In our binary translator, we emulate only status flags since the x86 integer instructions we emulate do not reference the value of control flags and system flags. We have emulated all of the status flags, including carry flag, parity flag, auxiliary flag, zero flag, sign flag, and overflow flag. While the executable is in execution, the values of these flags may be changed according to the execution result of an instruction.

There are two commonly used emulation strategies for the status flags, just-in-time emulation and lazy emulation. Just-in-time emulation refers to determine the flags value while emulating the instruction. This method may incur non-negligible overheads since the flags are emulated in every instruction. Lazy emulation refers to copy the information of the instruction and use it to determine the value of the demanded flags in further conditional instructions such as jz, js, jo, etc. By lazy emulation, the emulation has relative low overheads since only required flags are calculated. In our binary translator, we adopt both of the above two emulation methods. We use just-in-time emulation for flags required less calculations, including CF, AF, ZF, SF, and OF. The lazy emulation is applied for the time-consuming flag, PF. By the combination of these two methods, we may eliminate most of the emulation overheads of status flags.

E. Segment Register and Memory Addressing Mode Emulation

Our binary translator is developed in Linux operating system under x86 ISA. In protected mode, all memory accesses refer to a combination of a segment register (CS, DS, SS, or ES) and a logical address. The segment register points to the entry of the global descriptor table (GDT) or a local descriptor table (LDT) which stores the start address,

length, and privilege level of a segment. In Linux, all segment registers point to the same segment start address which is 0x00000000. Thus, in our binary translator, we preset the segment start address as 0x00000000 without accessing GDT or LDT.

In x86 ISA, a logical address can be expressed by several memory addressing modes, including direct, register indirect, based, indexed, based-indexed with no scale factor, and based-indexed with scale factor, etc. In LLVM MC definition, each x86 memory address is disassembled into five LLVM MC operands, including base register, scale factor, index register, displacement, and segment register. Each memory addressing mode of x86 ISA may be represented by an expression of these five operands, i.e., the memory address value is the sum of the segment register pointed value, the base register value, the product of index register value and scale factor, and the displacement.

F. Stack Initialization

When an ELF file is executed in Linux, the system loader will first parse the ELF file, set up the program segments and entry points, and initialize the process stack. After the loader completes the initialization, the program will be executed from the entry point. The auxiliary vector (auxv), environment vector, argument array, and argument count are pushed from higher memory addresses to lower memory addresses. All these pushed information are very important since they will be used at runtime. In our binary translator, we copy the information of environment vector, argument array, and argument count from the stack of our binary translator to the emulated stack. However, the auxiliary vector cannot be copied from the binary translator because the number of entries and values are different for different ISAs. If we just copy the auxiliary vector directly, errors may occur in the emulation since the wrong auxiliary entry is taken. In order to solve this problem, we write a small program to get all entries of the auxiliary vector on x86 ISA. Then, we may set up the emulated x86 auxiliary vector entries and values by calling system calls and use the information in our translator. For example, we emulate *AT_GID* by calling *getgid()* system call and emulate *AT_ENTRY* by obtaining the emulated program entry in our binary translator.

G. System Call Emulation

In Linux, when an x86 program is going to execute a system call, it must comply with the following steps:

- Put the system call number in EAX register.
- Set up the system call arguments in EBX, ECX, EDX, ESI, and EDI registers.
- Execute INT 80h system call instruction.
- Get the return value of the system call from EAX register.

The mapping table of x86 system call numbers and their corresponding Linux system calls is briefly shown in Table 1. For a system call, if the number of its parameters is less than or equal to five, these parameters may be put into the five registers, EBX, ECX, EDX, ESI, and EDI. Otherwise,

TABLE I. SYSTEM CALL NUMBERS AND ARGUMENTS IN INT 80.

System call # (%eax)	System call name	System call parameters				
		%ebx	%ecx	%edx	%esi	%edi
1	sys_exit	int				
2	sys_fork	struct_pt_regs				
3	sys_read	unsigned int	char *	size_t		
4	sys_write	unsigned int	const char *	size_t		
5	sys_open	const char *	int	int		
6	sys_close	unsigned int				
7	sys_waitpid	pid_t	unsigned int *	int		
...	...	...	...	...	...	...

these parameters are put into a related structure and passed by EBX which points to the head of the structure.

In our HTB, since LLVM IR does not provide system call instruction, x86 system calls are emulated by an external helper function. Once a system call INT 80h is disassembled to an LLVM MCInst, the LLVM MCInst will be translated to LLVM IR which calls the helper function with parameters including system call number and the emulated register values. In the helper function, a system call table is used to map the system call numbers to Linux provided system calls. And then, the helper function will call the corresponding Linux system call with the received parameters. Since x86 ISA has just few registers, some of the parameters are passed by structures in our design. Once the pointer of the structure is received, it is used to get the related structure values and call the corresponding Linux system call with these arguments.

H. Fast System Call Emulation

Traditionally, an x86 program uses INT 80h to execute a system call. While executing a system call in system kernel, the interrupt service routine will save the current system state and call the system call handler according to the system call number. Since a system call is executed in ring 0 and a user program is executed in ring 3, for each system call, the system must switch from ring 3 to ring 0, and then, after finishing the system call, switch back to ring 3. However, the switch between ring 0 and ring 3 may cause non-negligible overheads. Therefore, fast system call was introduced as a solution by Linux. Fast system call is a mechanism for speeding up the performance of a system call. In the execution, a fast system call uses *SYSENTER* and *SYSEXIT* instructions to eliminate the switch overhead between ring 0 and ring 3. For a program in execution, the program calls `__kernel_vsyscall` to execute the fast system call. The address of `__kernel_vsyscall` is saved in the `AT_SYSINFO` parameter of the auxiliary vector. After jumping to this address, the execution saves the program status, calls *SYSENTER* instruction, executes the system call, and then calls *SYSEXIT* to switch back to the user program.

However, not every target ISA supports fast system call. For an ISA other than x86, it might not have `AT_SYSINFO` in the auxiliary vector and does not support fast system call. In our design, we replace the content of `AT_SYSINFO` to an

allocated memory space entry and store INT 80h and RET in the allocated memory space, i.e., a fake fast system call is constructed. By this method, our binary translator may support the system calls of any target ISA no matter it supports fast system call or not. Furthermore, since we use x86 system call helper function, the Linux system call will follow the fast system mechanism if the target ISA supports fast system call; otherwise the system call will be executed in traditional way.

IV. EXPERIMENT RESULTS

Presently, our HBT is the first and the only hybrid binary translator which supports x86 source ISA. Since QEMU, a free dynamic binary translator, is widely used, easy to access, and able to run on most of the existed benchmarks, we compare our HBT with QEMU for performance evaluation.

A. Experiment Environments

In our experiment, the benchmark we used is 51 of the programs in EEMBC benchmark suite because our hybrid binary translator supports only source programs that are composed of x86 integer instructions currently. These source codes are compiled into 32-bit x86 executables by GCC 4.4.6 with `-msoft-float` and `-O2` optimization settings. The `-msoft-float` is software emulation for floating point instructions. All these executables are statically linked with the `uClibc 0.9.31` library [18]. We have two experiment environments, one is a 32-bit x86 target platform and another is a 64-bit x86 target platform. For the 32-bit target platform, it has an Intel Core2 Duo CPU E8400 with 3.0 GHz CPUs and 6144 KB cache per CPU, and the operating system is Debian GNU/Linux with 2.6.32 kernel. For the 64-bit target platform, it has an AMD Opteron Processor 6172 with 2.1 GHz CPUs and 512 KB cache per CPU, and the operating system is Debian GNU/Linux with 3.2.0 kernel. In our HBT, the LLVM version used is 3.1 for both 32-bit and 64-bit target platforms. The QEMU used in our experiments is version 1.5.1.

B. Performance Measurement

The experiments of x86-32 to x86-32 emulations and to x86-64 emulations have all been accomplished, and the results have the similar characteristics. Figure 5 illustrates the performance of 32-bit executable running on x86-64 machine and the performance results of our HBT and QEMU for x86-32 to x86-64 emulations. In this figure, the Y-axis is the normalized execution time over the native execution time of x86-64 executable. It is observed that most of the emulations, excluding those special cases, done by our HBT are 2.9 times slower than the native execution and 1.6 times faster than QEMU in average.

However, in these 51 programs of EEMBC benchmark, the emulation results of `cacheb01_lite.exe`, `canrdr01_lite.exe`, `canrdr816_lite.exe`, and `memacc816_lite.exe` in our HBT are much slower than that in QEMU, and the results of `autcor00data_1_lite.exe`, `fft00data_1_lite.exe`, and `ospf_lite.exe` are a little bit slower. According to our observation, the main reason why our HBT is slow for these

programs is because that the execution of these target executables switches between the statically translated code and the dynamic translator frequently, and these frequent context-switching incurs serious overheads. Since the x86 ISA has the characteristics of variable-length instruction format, interspersing data with instructions, and attaching pads for instruction alignment, the static-time translation of our HBT may incorrectly disassemble and translate instructions into target executable due to code discovery problem. When the incorrectly translated target executable is being executed, the execution will switch frequently between the statically translated code and the dynamic translator since the branch target cannot be found in the address mapping table generated at static-time translation. By our measurement, most of these overheads are come from context-switching.

In Figure 6, we show the execution time distribution for each target executable translated by our HBT being executed on x86-64 target machine. It is obvious that most of the programs spend most of their execution time in statically translated code. However, for the four programs of which the emulation performance is poor on our HBT, most of their execution time is consumed in dynamic translation, system call, and runtime system since they switch more frequently to the dynamic translator of our HBT to perform dynamic translation than the other programs.

C. Code Size of Target Executable

In a target executable, the data and text sections of source executable are keep in the source image and the translated text section is in the text section. In our measurement for the code size of each target executable, we have recorded the sizes of data section and text section of the source executable and the size of the translated text section. According to the collected data, the text sections of the target executables are about 3 to 5 times larger than that of the source executables.

V. CONCLUSIONS AND FUTURE WORKS

A hybrid binary translator may translate the source executable with aggressive optimizations at static time, and the execution of the target executable will switch to its dynamic translator when the target address of an indirect branch cannot be found in the address mapping table constructed statically. Therefore, a hybrid binary translator has the advantage of performance and may handle indirect branches correctly. In this paper, we successfully apply the LLVM infrastructure to implement a hybrid binary translator which support x86 source ISA. In the experiments, we translate x86-32 source executable to x86-32 and x86-64 target executables. By comparing our experiment result with that of QEMU, our hybrid translator is about 1.6 times faster for most of the programs of EEMBC.

In our implementation of the x86 front-end supported hybrid binary translator, we use LLVM MC tool kit to translate x86 machine code to LLVM MCInst and translate more than six hundreds of LLVM MCInsts into LLVM IR. Besides instruction translation, we also emulate the stack, system call, and fast system call of the source executable. Finally, we add the support for x86-64 target ISA. We

believe that issues we encountered in our implementation may be a good reference for researchers who intend to make a binary translator with the characteristics of hybrid translation and CISC ISA supported front-end.

By observing our experiment results, the emulation performance of some programs in EEMBC is poor. This is because that our hybrid binary translator incorrectly disassembles and translates some of the source executable at static time. When a target executable which is not completely translated statically is running, the execution may switch between the statically translated code and the dynamic translator frequently. In the future, we will study the possible methods to reduce the overheads of context switches, for examples, modifying the execution flow of the hybrid translated target executable, combining the statically generated address mapping table with that of the dynamic binary translator, etc. Currently, our x86 front-end supported hybrid binary translator can successfully translate x86 source executable that consists of integer instructions only. The support of x86 float point instructions and SIMD instructions will be implemented in the future. Moreover, we will also implement the supports for other target ISAs, for example, ARM, for revealing the retargetability of our HBT.

ACKNOWLEDGMENT

The authors would like to thank the Ministry of Science and Technology of the Republic of China for financially supporting this research under Contract No. NSC 100-2218-E-009-012-MY3 and MOST 103-2221-E-009-104-MY3.

REFERENCES

- [1] R. L. Sites, A. Chernoff, M. B. Kirk, M. P. Marks and S. G. Robinson, "Binary Translation," *Communications of the ACM*, vol. 36, no. 2, pp. 69-81, Feb. 1993.
- [2] E. R. Altman, D. Kaeli and Y. Sheffer, "Welcome to the opportunities of binary translation," *Computing & Processing*, vol. 33, no. 3, pp. 40-45, Mar. 2000.
- [3] B. Schwarz, S. Debray and G. Andrews, "A link-time optimizer for the Intel IA-32 architecture," in *2001 Workshop on Binary Translation (WBT-2001)*, 2001.
- [4] B. D. Sutter, L. V. Put, D. Chanet, B. D. Bus and K. D. Bosschere, "Link-time compaction and optimization of ARM executables," *ACM Transactions on Embedded Computing Systems*, vol. 6, no. 1, Feb. 2007.
- [5] A. Chernoff, M. Herdeg, R. Hookway, C. Reeve, N. Rubin, T. Tye, S. B. Yadavalli and J. Yates, "A Profile-Directed Binary Translator," *IEEE Micro*, vol. 18, pp. 56-64, 1998.
- [6] J. Smith and R. Nair, *Virtual Machines: Versatile Platforms for Systems and Processes*, Morgan Kaufmann, 2005.
- [7] K. Ebcioğlu, "Dynamic binary translation and optimization," *IEEE Transactions on Computers*, vol. 50, no. 6, pp. 529 - 548, Jun. 2001.
- [8] B.-Y. Shen, J.-Y. You, W. Yang and W.-C. Hsu, "An LLVM-based Hybrid Binary Translation System," in *International Symposium on Industrial Embedded Systems (SIES'12)*, Karlsruhe, Germany, 2012.
- [9] A. Chernoff, M. Herdeg, R. Hookway, C. Reeve, N. Rubin, T. Tye, S. B. Yadavalli and J. Yates, "FX!32: A Profile-Directed Binary Translator," *IEEE Micro*, vol. 18, pp. 56-64, 1998.
- [10] L. Baraz, T. Devor, O. Etzion, S. Goldenberg, A. Skaletsky, Y. Wang and Y. Zemach, "IA-32 Execution Layer: a two-phase dynamic translator designed to support IA-32 applications on Itanium®-based systems," in *Proceedings of the 36th annual IEEE/ACM International Symposium on Microarchitecture*, p. 191, 2003.

- [11] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," in *USENIX 2005 Annual Technical Conference*, 2005.
- [12] D.-Y. Hong, C.-C. Hsu, P.-C. Yew, J.-J. Wu, W.-C. Hsu, C.-M. Liu and Y.-C. Chung, "HQEMU: a multi-threaded and retargetable dynamic binary translator on multicores," in *Proceedings of the Tenth International Symposium on Code Generation and Optimization*, New York, 2012.
- [13] C. Lattner and V. Adve, "LLVM: a compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization*, 2004.
- [14] "Clang: A C Language Family Frontend for LLVM," [Online]. Available: <http://clang.llvm.org/>. [Accessed 27 6 2013].
- [15] "The LLDB Debugger," [Online]. Available: <http://lldb.llvm.org/>. [Accessed 27 6 2013].
- [16] "DragonEgg - Using LLVM as a GCC backend," [Online]. Available: <http://dragonegg.llvm.org/>. [Accessed 27 6 2013].
- [17] "Intro to the LLVM MC Project," 9 Apr. 2010. [Online]. Available: <http://blog.llvm.org/2010/04/intro-to-llvm-mc-project.html>.
- [18] "uClibc," [Online]. Available: <http://www.uclibc.org/>. [Accessed 3 6 2013].

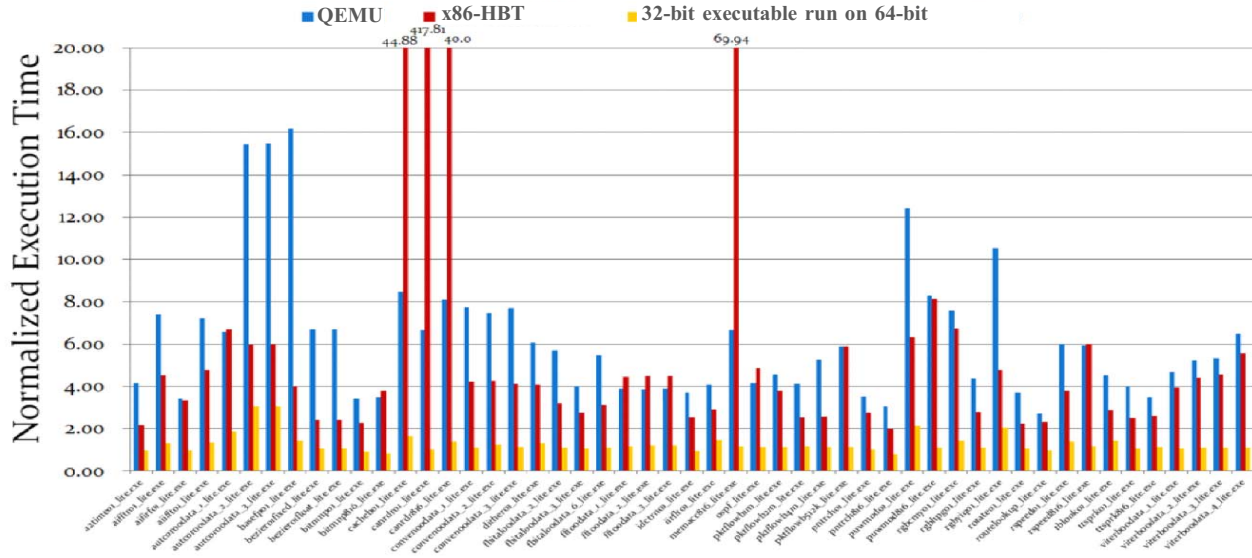


Figure 5. Normalized execution time for x86-32 to x86-64 emulations on x86-64 target machine.

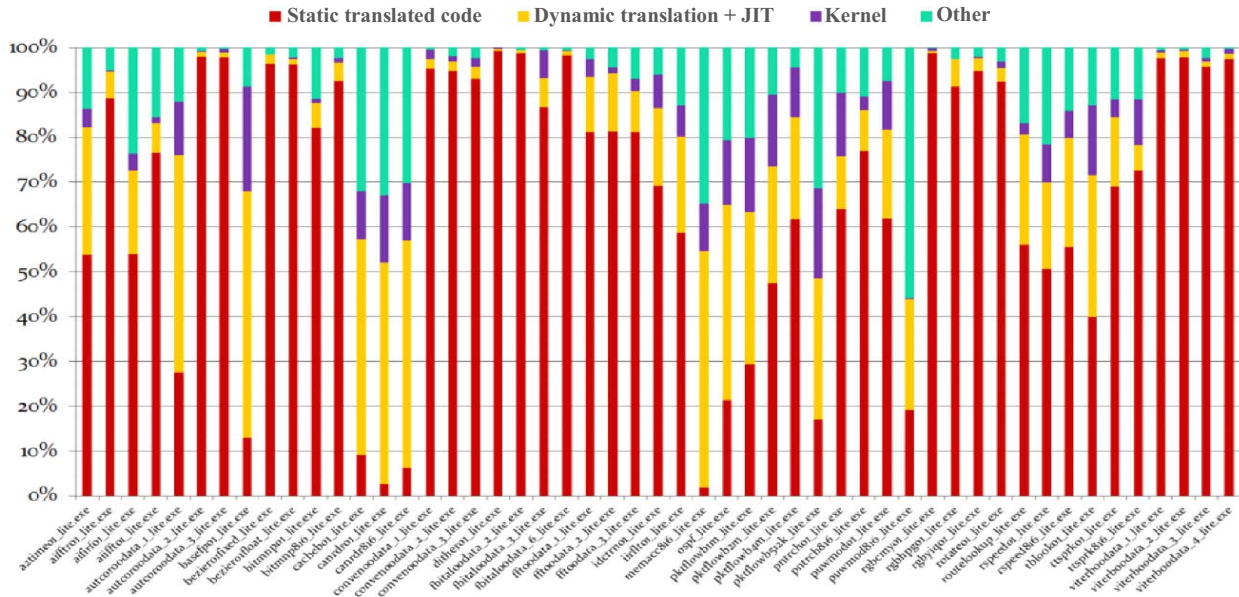


Figure 6. Execution Time Distribution for x86-64 target executables generated by our HBT.